

El entorno java para la programación algoritmos

M4: Fundamentos de programación en Java

| **AE3:** Utilizar la sintaxis básica del lenguaje Java para la construcción de programas que resuelven un problema de baja complejidad

Introducción

En esta lección descubrirás cómo trabajar con variables en Java y a manipular los distintos tipos de datos que posee.

A medida que avancemos, conocerás cómo realizar diversas operaciones, desde simples cálculos matemáticos hasta comparaciones lógicas. También aprenderás sobre expresiones, sentencias y bloques de código, elementos fundamentales para estructurar tu programa de manera clara y ordenada.

En el mundo de la programación, la toma de decisiones es crucial. Por esto, pondrás en práctica el uso de sentencias condicionales y repetitivas, que te permitirán ejecutar diferentes acciones en función de ciertas condiciones y te facilitarán la tarea de realizar operaciones de manera iterativa.

Para manejar conjuntos de datos, aprenderás sobre arreglos y colecciones, que te permitirán almacenar y manipular múltiples elementos y trabajar con cadenas de texto de manera eficiente.

A lo largo de esta lección, adquirirás las habilidades necesarias para desarrollar programas básicos en Java de forma estructurada y eficaz.

Aprendizaje esperado

Al finalizar la lección podrás:

- Utilizar la sintaxis básica del lenguaje Java para la construcción de programas que resuelven un problema de baja complejidad.

Variables en Java

Java es un lenguaje de tipado estático, por lo cual **todas las variables deben ser declaradas antes de ser utilizadas**. Estas tendrán un tipo de dato definido (ya sea un tipo de dato primitivo o una clase) y un nombre de identificador. El tipo de dato se asignará a la hora de definir la variable.

```
int numero = 2;
```

```
String cadena = "Hola";
```

```
double decimal = 2.4;
```

```
boolean flag = true;
```

El operador ' = ' funciona para asignarle un valor a cada variable. Esto se conoce como inicialización. En la mayoría de los casos es necesario inicializar la variable al momento de crearla, pero no es obligatorio.

Tipos de datos primitivos

Estos tipos de datos son predefinidos por el lenguaje. La biblioteca Java proporciona clases asociadas a estos tipos que proporcionan métodos que facilitan su manejo.

byte: como su propio nombre denota, emplea un solo byte (8 bits) de almacenamiento. Esto permite almacenar valores en el rango [-128 , 127].

short: usa el doble de almacenamiento que el anterior, lo cual hace posible representar cualquier valor en el rango [-32.768, 32.767].

int: emplea 4 bytes de almacenamiento y es el tipo de dato entero más empleado. El rango de valores que puede representar va de -2,147,483,648 a 2,147,483,647.

long: es el tipo entero de mayor tamaño, 8 bytes (64 bits), con un rango de valores desde -9,223,372,036,854,775,808 a 9,223,372,036,854,775,807.

float: conocido como tipo de precisión simple, emplea un total de 32 bits. Con este tipo de datos es posible representar números en el rango de 1.4E-45 a 3.4028235E38. Esto significa que puede almacenar números en el rango negativo y positivo, incluyendo valores decimales, pero con una precisión limitada debido a su naturaleza de precisión simple.

double: se utiliza para representar números de punto flotante de precisión doble. Sigue un esquema de almacenamiento similar al anterior, pero usando 64 bits en lugar de 32. Esto le permite representar valores en el rango de 4.9E-324 a 1.7976931348623157E308..

El tipo double ofrece un rango de representación más amplio y una mayor precisión en comparación con el tipo float. Sin embargo, también ocupa más espacio de memoria. Por lo tanto, es importante seleccionar el tipo de dato adecuado según las necesidades específicas de tu programa.

char: el tipo de dato caracter es un simple caracter unicode de 16 bits. Su valor mínimo es de '\u0000' (En entero es 0) y su valor máximo es de '\uffff' (en entero es 65,535). Los datos de tipo caracter en Java se escriben entre comillas simples, por ejemplo 'a', o también indicando su valor Unicode, por ejemplo '\u0061'.

String: Además de los tipos de datos primitivos el lenguaje de programación Java provee también un soporte especial para cadenas de caracteres a través de la clase String.

Encerrando la cadena de caracteres con comillas dobles se creará de manera automática una nueva instancia de un objeto tipo String.

String cadena = "Hola";

Los objetos String son inmutables, esto significa que una vez creados, sus valores no pueden ser cambiados. Si bien esta clase no es técnicamente un tipo de dato primitivo, el lenguaje le da un soporte especial para que actúe como tal.

¿Cómo se ve en Java la declaración de una variable + tipo de dato?

```
12 //Este es el metodo Main.
13 public static void main(String[] args) {
14
15     String nombre;
16     int numero;
17     double decimales;
18 }
```

¡Manos a la obra! 🛠️

1. Crear un proyecto de Java en Eclipse y definir al menos 6 variables utilizando distintos tipos de datos.

Encontrarás la solución al final de todo en el apartado “Resoluciones Desafíos”. Te aconsejamos mirarla solo cuando hayas terminado.

Operadores

Los **operadores aritméticos** en Java son aquellos que nos permiten realizar operaciones matemáticas: **suma, resta, multiplicación, división y resto**.

Operador	Descripción
+	Suma matemática. Concatena cadenas para la suma de String
-	Operador de Resta
*	Operador de Multiplicación
/	Operador de División
%	Operador de Resto

Los **operadores unarios** en Java son aquellos que solo requieren un operando para funcionar.

Operador	Descripción
+	Unario suma. Indica un número positivo.
-	Unario resta. Niega una expresión.
++	Incremento. Incrementa el valor en 1.
--	Decremento. Decrementa el valor en 1.
!	Complemento lógico. Invierte el valor de un booleano.

Los operadores de **igualdad y relación** son aquellos que nos permiten **comparar** el contenido de una variable contra otra, atendiendo si son variables con un valor igual o distinto, o bien si los valores son mayores o menores.

Operador	Descripción
==	igual a

<code>!=</code>	no igual a (distinto de)
<code>></code>	mayor que
<code>>=</code>	mayor o igual que
<code><</code>	menor que
<code><=</code>	menor o igual que

Los **operadores lógicos** son aquellos que evalúan dos expresiones booleanas.

Operador	Descripción
<code>&&</code>	Operador condicional AND ("Y")
<code> </code>	Operador condicional OR ("O")
<code>?:</code>	Operador Ternario
<code>instanceof</code>	Operador instanceof

Expresiones, sentencias y bloques !

Un programa en Java se compone de un conjunto de sentencias que se ejecutan para resolver un problema. Las sentencias son el elemento básico de ejecución de los programas Java, donde también encontraremos expresiones y bloques.

Expresiones

Una expresión es un conjunto de variables, operadores e invocaciones de métodos que se construyen para poder ser evaluadas retornando un resultado.

```
int valor = 1;
```

```
if (valor 1 > valor2) { ... }
```

Cuando tengamos expresiones de evaluación complejas es recomendable que utilicemos paréntesis para saber cual es el orden de ejecución de operaciones. En el caso de no utilizar paréntesis se ejecutará el orden de preferencia de operadores. Por ejemplo, la división tiene más preferencia que la suma.

Sentencias

Una sentencia es la unidad mínima de ejecución de un programa. Un programa se compone de uno o varios conjuntos de sentencias que acaban resolviendo un problema. **Al final de cada una de las sentencias encontraremos un punto y coma (;).**

- Sentencias de **declaración**: `int valor = 2;`
- Sentencias de **asignación**: `valor = 2;`
- Sentencias de **incremento o decremento**: `valor++;`
- Invocaciones a **métodos**: `System.out.println("Hola Mundo");`
- Creaciones de **objetos**: `Circulo miCirculo = new Circulo(2,3);`
- Sentencias de **control de flujo**: `if (valor>1) { ... };`

Bloques

Un bloque es un conjunto de sentencias que están delimitadas por llaves.

```
if (expresion) {  
    // Bloque 1  
} else {  
    // Bloque 2  
}
```

Sentencias condicionales ✨

Los condicionales **son estructuras de control que cambian el flujo de ejecución de un programa de acuerdo a si se cumple o no una condición**. Cuando el flujo de control del programa llega al condicional, el programa evalúa la condición y determina el camino a seguir. Existen dos tipos de estructuras condicionales, las estructuras **if / else** y la estructura **switch**.

If - else

La estructura **if** es la más básica de las estructuras de control de flujo. Esta estructura le indica al programa que ejecute determinado bloque del código si y sólo si la condición evaluada es verdadera («true»).

La forma más simple de esta estructura es la siguiente:

```
if(<condición>){
    <sentencias>
}
```

En esta estructura, **<condición>** es una expresión condicional cuyo resultado luego de la evaluación es un dato **booleano**(lógico) verdadero o falso. El bloque de instrucciones **<sentencias>** se ejecuta si y sólo si la expresión se evalúa como **true**, es decir, se cumple la condición.

En caso de que la condición no se cumpla y se necesite ejecutar otro bloque de código para estos casos, otra forma de expresar esta estructura es la siguiente:

```
if(<condición>){
    <sentencias A>
} else {
    <sentencias B>
}
```

El flujo de control del programa funciona de la misma manera, cuando se ejecuta la estructura **if**, se evalúa la expresión condicional, si el resultado de la condición es verdadero se ejecutan las sentencias que se encuentran contenidas dentro del bloque de código **if (<sentencias A>)**.

Si la evaluación da como resultado **false**, se ejecutan las sentencias contenidas dentro del bloque **else (<sentencias B>)**.

En muchas ocasiones, se **anidan** estructuras alternativas if-else, de forma que se pregunte por una condición si anteriormente no se ha cumplido otra y así sucesivamente.

```
if (<condicion1>) {
    <sentencias A>
} else if(<condicion2>){
    <sentencias B>
} else {
```



```
<sentencias C>  
}
```

Switch

El bloque **switch** evalúa qué valor tiene la variable, y de acuerdo con el valor que posee ejecuta las sentencias del bloque **case** correspondiente.

```
switch(<variable>){  
    case <valor1>:  
        <sentencias1>  
        break;  
    case <valor2>:  
        <sentencias2>  
        break;  
    default:  
        <sentencias3>  
}
```

El uso de la sentencia **break** que va luego de cada case tiene la función de detener la sentencia switch que la envuelve, es decir que el control de flujo del programa continúa con la primera sentencia que se encuentra a continuación del cierre del bloque switch.

Si el programa comprueba que se cumple el primer valor (**valor1**) se ejecutará el bloque de instrucciones **<sentencias1>**.

Las instrucciones dentro del bloque **default** se ejecutan cuando la variable que se está evaluando **no coincide con ninguno de los valores case**. Esta sentencia equivale al else de la estructura if-else.

Sentencias repetitivas

Las estructuras que repiten una secuencia de instrucciones un número determinado de veces se denominan **bucles**, y se denomina **iteración** al hecho de repetir la ejecución de una secuencia de acciones.

Todo bucle se encuentra asociado a una condición, que es la que va a determinar cuándo y cuántas veces se repite.

While (mientras)

Esta estructura de control repite un conjunto de instrucciones **mientras una condición se cumpla (true)**, en cuanto la condición no se cumple el bucle deja de ejecutarse. Es por ello que dentro del bloque deberán existir sentencias que **modifiquen la evaluación** de la expresión, ya que de no hacerse se podría entrar en un bucle infinito.

En el caso de que la condición se evalúe por primera vez como **false**, **el ciclo no será ejecutado ni una vez**.

```
while (<condición>) {  
    <sentencias>;  
}
```

Do - While (repetir)

En el caso de la estructura repetitiva **do-while** el funcionamiento es casi el mismo que el de while. Pero con una diferencia, **primero se ejecuta el bloque de sentencias y luego se evalúa la expresión**. Por lo tanto, siempre se ejecutará al menos una vez.

```
do{  
    <sentencias>;  
}while (<condición>;
```

Al igual que en el while, en el bloque de sentencias deberemos modificar la condición para poder salir del bucle.

For (para)

La estructura **for** proporciona una forma compacta de recorrer un rango de valores **cuando la cantidad de veces que se debe iterar un bloque de código es conocida**.

```
for (<inicialización>; <terminación>; <incremento>) {  
  
<sentencias>;  
  
}
```

La expresión **<inicialización>** inicializa el bucle y se ejecuta una sola vez al comienzo de la ejecución del bucle. El bucle termina cuando al evaluar la expresión **<terminación>** el resultado que se obtiene es **false**.

La expresión **<incremento>** se invoca después de cada iteración que realiza el bucle; esta expresión incrementa o decrementa un valor hasta que se cumpla la condición de **<terminación>** del bucle.

Como regla general podemos afirmar que se utilizará el **bucle for** cuando se conozca de antemano el número exacto de veces que debe repetirse un determinado bloque de instrucciones.

Se utilizará el **bucle do-while** cuando no se conoce exactamente el número de veces que se ejecutará el bucle, pero se sabe que por lo menos se ha de ejecutar una vez.

Se utilizará el **bucle while** cuando es posible que no deba ejecutarse ninguna vez.

Sentencias de salto

En Java existen dos formas de realizar un salto incondicional en el flujo normal de un programa: las instrucciones **break** y **continue**.

Break

La instrucción **break** se utiliza para abandonar una estructura de control, tanto de las condicionales (if-else y switch) como de las repetitivas (for, do-while y while). En el momento que se ejecuta la instrucción **break**, el control del programa sale de la estructura en la que se encuentra contenida y continúa con la ejecución secuencial del programa.

Continue

La sentencia **continue** detiene la iteración en el punto del programa en donde se encuentra el continue, pero en lugar de salir del bucle, continúa con la siguiente iteración. La instrucción continue transfiere el control del programa desde la instrucción continue directamente a la cabecera del bucle (for, do-while o while) donde se encuentra.

Arreglos y Colecciones

Los arreglos permiten almacenar varios objetos de diferentes tipos; y también permiten acceder a un valor específico según su posición en una tabla. El acceso a los elementos de un arreglo es extremadamente eficiente debido a su estrecha relación con la organización interna de la memoria de una computadora. Los valores en un arreglo están dispuestos en un orden determinado, reflejando su ubicación en la memoria física. Por lo tanto, el acceso se gestiona directamente a nivel de hardware.

Al igual que la declaración de otros tipos de variables, la declaración de un arreglo tiene dos componentes: el tipo de dato del arreglo y su nombre. El tipo de dato del arreglo se escribe como `tipo[]`, en donde, tipo es el tipo de dato de cada elemento contenido en él. Los corchetes sirven para indicar que esa variable va a ser un arreglo. El tamaño del arreglo no es parte de su tipo (es por esto que los corchetes están vacíos).

Una vez declarado un arreglo hay que crearlo/dimensionarlo, es decir, hay que asignarle un tamaño para almacenar los valores (y que nunca va a cambiar). La creación de un arreglo se hace con el operador **new**. Recordemos que las matrices son bidimensionales por lo que tienen dos tamaños, uno para las filas y otro para las columnas de la matriz.

Declaración y creación de un Arreglo o Vector.

```
tipo[] arregloV = new tipo[Tamaño];
```

Declaración y creación de una Matriz

```
tipo[][] arregloM = new tipo[Filas][Columnas];
```

```

14 public static void main(String[] args) {
15
16     // Creo un arreglo llamado vector con dimension 5
17     // solo puedo alojar numeros enteros en el
18     int[] vector = new int[5];
19
20     // Creo una matriz con dimension 3x3
21     // solo puedo alojar cadenas en ella
22     String[][] matriz = new String[3][3];
23 }
24
25
26 }

```

Expandiremos la explicación y podrás ver más ejemplos prácticos en los videos disponibles-

Colecciones

Una colección representa **un grupo de objetos**, conocidos como **elementos**. Podemos crear una colección con cualquier tipo de objeto, incluso los creados por nosotros. La única restricción es que no podremos crear una colección sólo de tipo primitivo, para eso usaremos los tipos de objetos equivalentes a los primitivos. Por ejemplo: en vez de int, hay que utilizar **Integer**. A continuación veremos los objetos equivalentes a los primitivos de Java.

Tipos de datos	
Primitivos	Objetos
int	Integer
double	Double
long	Long
char	Character
boolean	Boolean
String ya es un objeto, por lo que no tiene tipo primitivo	

La principal **diferencia** entre las colecciones y los arreglos tradicionales es que las colecciones crecen de manera dinámica a medida que se van agregando objetos. No necesitamos saber de antemano la cantidad de elementos con la que vamos a trabajar.

Vamos a expandir la explicación de Colecciones cuando lleguemos a la lección de Programación Orientada a Objetos.

Clase de utilidad: String

La clase String en Java es una herramienta poderosa para manipular cadenas de texto. Con ella puedes concatenar cadenas para unir las, obtener la longitud de una cadena, buscar subcadenas, reemplazar partes de una cadena y mucho más. Además, la clase String proporciona métodos para comparar cadenas y realizar operaciones de formato.

Principales métodos de la clase String

Método	Tipo de dato	Descripción
charAt(int index)	Char	Devuelve el caracter del índice especificado
compareTo(String otraCadena)	Entero	Compara dos cadenas entre sí
concat(String str)	Cadena	Une la cadena especificada al final de nuestra cadena
equals(Objeto unObjeto)	Booleano	Compara la cadena con el objeto especificado

indexOf(String str)	Entero	Devuelve el índice dentro de la cadena de la primera coincidencia de la subcadena especificada
length()	Entero	Devuelve la cantidad de caracteres de la cadena
replace(char viejo, char nuevo)	Cadena	Devuelve una nueva cadena del reemplazo de todas las coincidencias del caracter viejo con el caracter nuevo
substring(int inicio)	Cadena	Devuelve una nueva cadena que es una subcadena de la original
substring(int inicio, int final)	Cadena	Devuelve una nueva cadena que es una subcadena de la original delimitada por el inicio y final informados
toLowerCase()	Cadena	Convierte toda la cadena a minúsculas usando las reglas locales del sistema
toString()	Cadena	Se devuelve el objeto en sí.

toUpperCase()	Cadena	Convierte toda la cadena a mayúsculas usando las reglas locales del sistema
trim()	Cadena	Devuelve una copia de la cadena donde se omiten los espacios
valueOf(tipo de dato primitivo x)	Cadena	Devuelve la representación de cadena del argumento de tipo de datos pasado.

Clase de utilidad: Math

Con la clase Math **puedes realizar operaciones aritméticas básicas**, como sumar, restar, multiplicar y dividir números. También puedes utilizarla para realizar cálculos más complejos, como redondear números, calcular raíces cuadradas, generar números aleatorios y mucho más.

Además de las operaciones matemáticas, la clase Math también contiene constantes matemáticas útiles, como PI (π) y E (número de Euler). Estas constantes te permiten realizar cálculos precisos y utilizar valores estándar en tus programas.

Para invocar los siguientes métodos, simplemente debes colocar la palabra Math. y a continuación el nombre del método que quieres utilizar.

Principales métodos de la clase Math

Método.	Descripción.
abs(tipo de dato numérico a)	Devuelve el valor absoluto de un valor numérico introducido como parámetro.

<code>cbrt(double a)</code>	Devuelve la raíz cúbica de un valor double.
<code>cos(double a)</code>	Devuelve el coseno trigonométrico de un ángulo.
<code>exp(double a)</code>	Devuelve el número <i>e</i> de Euler elevado a la potencia de un valor double.
<code>log(double a)</code>	Devuelve el logaritmo natural (base <i>e</i>) de un valor double.
<code>log10(double a)</code>	Devuelve el logaritmo de base 10 de un valor double.
<code>max(double a, double b)</code>	Devuelve el mayor de dos valores double
<code>max(int a, int b)</code>	Devuelve el mayor de dos valores Enteros.
<code>min(double a, double b)</code>	Devuelve el menor de dos valores double.
<code>min(int a, int b)</code>	Devuelve el menor de dos valores enteros.
<code>pow(double a, double b)</code>	Devuelve el valor del primer argumento elevado a la potencia del segundo argumento.
<code>random()</code>	Devuelve un valor double con un signo positivo, mayor o igual que 0.0 y menor que 1.0.
<code>round(float a)</code>	Devuelve el int más cercano y redondeado al float introducido.

<code>sin(double a)</code>	Devuelve el seno trigonométrico de un ángulo.
<code>sqrt(double a)</code>	Devuelve la raíz cuadrada positiva correctamente redondeada de un valor double.
<code>tan(double a)</code>	Devuelve la tangente trigonométrica de un ángulo.

Constante.	Descripción.
E	Devuelve el valor más aproximado del número e
PI	Devuelve el valor más aproximado del número PI.

El método **Math.random** se utiliza para generar números al azar. El rango o margen con el que trabaja el método random oscila entre 0.0 y 1.0 (Este último no incluido).

Por lo tanto, para generar un número entero entre 0 y 9, Hay que escribir la siguiente sentencia:

```
int numero = (int) (Math.random () * 10);
```

De esta manera al multiplicar el valor por 10, el rango de valores posibles se convierte en: $0.0 \leq \text{numero} < 10.0$

Cierre

En esta lección exploramos los fundamentos de la programación en Java. Hemos aprendido sobre las variables y cómo utilizarlas para almacenar y manipular datos en nuestros programas. Además, hemos descubierto los diferentes tipos de datos que nos permiten trabajar con números, caracteres y valores booleanos.

Las sentencias condicionales y repetitivas nos han permitido controlar el flujo de ejecución de nuestros programas, tomando decisiones y repitiendo acciones según determinadas condiciones. Estas estructuras nos brindan flexibilidad y control sobre la lógica de nuestros sistemas.

Además, hemos explorado los arreglos y las colecciones, que nos permiten almacenar conjuntos de datos de manera organizada y acceder a ellos de forma eficiente. También hemos descubierto las operaciones básicas que podemos realizar con las clases de utilidad String y Math, para manipular y trabajar con cadenas de caracteres y valores numéricos.

¡Nos vemos de nuevo, más adelante! 

Referencias

- ManualWeb.net. (s.f.). Variables en Java. Recuperado de <https://www.manualweb.net/java/variables-java/>
- OpenWebinars. (s.f.). Tipos de datos (Java). Recuperado de <https://www.openwebinars.net/temas/java>
- ManualWeb.net. (s.f.). Expresiones, sentencias y bloques en Java. Recuperado de <https://www.manualweb.net/java/expresiones-sentencias-bloques-java/>
- Disco Duro de Roer. (2013, 18 de septiembre). Estructuras condicionales en Java. Recuperado de <https://www.discoduroderoer.es/tag/condicional/>
- Garro, A. (s.f.). Sentencias repetitivas o bucles (for, while, do-while). Recuperado de La Web del Programador: Guía JAVA <https://www.lawebdelprogramador.com/cursos/java/10494-Guia-JAVA.html>
- ManualWeb.net. (s.f.). Arreglos en Java. Recuperado de <https://www.manualweb.net/java/arreglos-java/>
- Jcodepoint. (s.f.). Clase String en Java. Recuperado de <https://jcodepoint.com/clase-string>
- Oregoom. (s.f.). Clase Math en Java. Recuperado de <https://www.oregoom.com/clase-math-en-java/>

¡Muchas gracias!

Nos vemos en la próxima lección

